

LAB ASSIGNMENT REPORT

MODULE 3 – PYTHON BDD & RESTFUL AUTOMATIONS

Name: Saheli Deb

Enrollment No.: 12023052019071

Course: Python Automation

Report Type: Exact Syllabus-wise Practical / Lab Report

Module 3 Lab Coverage

This practical report follows the prescribed Module 3 headings: 3.1 Python Basics for REST Automation, 3.2 Python HTTP Libraries, 3.3 API Automation and Database Integration, 3.4 Handling HTTP Responses, and 3.5 BDD with Behave.

Environment: Python 3, Requests, PyTest, Behave and SQLite. Install with: **`python -m pip install requests pytest behave`**.

3.1 Python Basics for REST Automation

Python functions, collections, JSON-style dictionaries and assertions are used to build reusable REST validation logic.

Experiment 1 – Python Functions and JSON Validation

Aim: To create reusable validation logic for API response data.

Procedure

1. Define a validation function.
2. Check required JSON fields.
3. Return a Boolean result.
4. Assert the result using sample API data.

Source Code

```
def validate_post(data):
    required = ["id", "title", "userId"]
    for key in required:
        if key not in data:
            return False
    return data["id"] > 0 and bool(data["title"])

sample = {"id": 1, "title": "REST Automation", "userId": 1}

assert validate_post(sample)
print("API data validation passed")
```

Expected Result: All required fields are present and the validation passes.

Conclusion: Functions encapsulate reusable API validation logic.

Experiment 2 – Lists, Dictionaries and Iteration

Aim: To filter a collection of API-style JSON records.

Procedure

1. Create a list of dictionaries.
2. Iterate/filter by userId.
3. Print matching records.
4. Validate the count.

Source Code

```
posts = [
    {"id": 1, "title": "Login API", "userId": 1},
    {"id": 2, "title": "POST API", "userId": 1},
    {"id": 3, "title": "BDD Test", "userId": 2}
]

user_one = [p for p in posts if p["userId"] == 1]

print(user_one)
assert len(user_one) == 2
```

Expected Result: Two records for user 1 are selected.

Conclusion: Python collections are useful for response-data validation.

3.2 Python HTTP Libraries

The Requests library is used to perform HTTP operations and inspect REST responses.

Experiment 3 – GET Request Using Requests

Aim: To retrieve and validate a REST resource.

Procedure

1. Import requests.
2. Send GET with a timeout.
3. Check status code.
4. Parse JSON.
5. Validate the ID.

Source Code

```
import requests

response = requests.get(
    "https://jsonplaceholder.typicode.com/posts/1",
    timeout=10
)

print("Status:", response.status_code)
data = response.json()

assert response.status_code == 200
assert data["id"] == 1
```

Expected Result: HTTP 200 is received and the returned object has ID 1.

Conclusion: Requests provides a simple Python interface for REST automation.

Experiment 4 – POST with JSON and Headers

Aim: To create a resource using JSON data and HTTP headers.

Procedure

1. Prepare headers and payload.
2. Send POST.
3. Validate 201 Created.
4. Validate returned fields.

Source Code

```
import requests

headers = {
    "Accept": "application/json",
    "Content-Type": "application/json"
}

payload = {
    "title": "Module 3 Lab",
    "body": "Python REST automation",
    "userId": 1
}

response = requests.post(
    "https://jsonplaceholder.typicode.com/posts",
    json=payload,
    headers=headers,
    timeout=10
)

assert response.status_code == 201
assert response.json()["title"] == payload["title"]
print("POST successful")
```

Expected Result: The API returns 201 and the response contains the submitted title.

Conclusion: POST automation requires correct payload and response validation.

3.3 API Automation and Database Integration

REST results can be stored in or compared with a database to verify backend data integrity.

Experiment 5 – Store API Response in SQLite

Aim: To save selected API response fields in SQLite.

Procedure

1. Create a table.
2. Call the API.
3. Extract JSON fields.
4. Insert them into SQLite.
5. Query and compare the row.

Source Code

```
import sqlite3
import requests

response = requests.get(
    "https://jsonplaceholder.typicode.com/posts/1",
    timeout=10
)
assert response.status_code == 200
post = response.json()

conn = sqlite3.connect("api_results.db")
cur = conn.cursor()

cur.execute("""
CREATE TABLE IF NOT EXISTS posts (
    id INTEGER PRIMARY KEY,
    title TEXT,
    user_id INTEGER
)
""")

cur.execute(
    "INSERT OR REPLACE INTO posts VALUES (?, ?, ?)",
    (post["id"], post["title"], post["userId"])
)
conn.commit()

cur.execute(
    "SELECT id, title, user_id FROM posts WHERE id = ?",
    (post["id"],)
)
row = cur.fetchone()

assert row[0] == post["id"]
assert row[2] == post["userId"]
print("Database validation passed")

conn.close()
```

Expected Result: The API record is stored and the queried database values match.

Conclusion: API-to-database validation supports backend data integrity testing.

Experiment 6 – Compare API Data with Database Data

Aim: To compare an API field with an expected value stored in SQLite.

Procedure

1. Create an expected-data table.
2. Insert an expected title.
3. Call the API.
4. Read expected data.
5. Compare values.

Source Code

```
import sqlite3
import requests

conn = sqlite3.connect(":memory:")
cur = conn.cursor()

cur.execute(
    "CREATE TABLE expected_posts (id INTEGER, title TEXT)"
)
cur.execute(
    "INSERT INTO expected_posts VALUES (?, ?)",
    (1, "sunt aut facere repellat provident occaecati")
)
conn.commit()

api = requests.get(
    "https://jsonplaceholder.typicode.com/posts/1",
    timeout=10
)
assert api.status_code == 200

cur.execute(
    "SELECT title FROM expected_posts WHERE id = ?",
    (1,)
)
expected = cur.fetchone()[0]

assert api.json()["title"] == expected
print("API and database values match")

conn.close()
```

Expected Result: The API title matches the expected database value.

Conclusion: Database comparison validates backend consistency.

3.4 Handling HTTP Responses

Response handling includes status-code validation, JSON parsing, HTTP errors, timeouts and exception management.

Experiment 7 – Successful and Error Responses

Aim: To validate both positive and negative API behavior.

Procedure

1. Request an existing resource.
2. Assert 200.
3. Request a non-existent resource.
4. Assert 404.

Source Code

```
import requests

ok = requests.get(
    "https://jsonplaceholder.typicode.com/posts/1",
    timeout=10
)
assert ok.status_code == 200

missing = requests.get(
    "https://jsonplaceholder.typicode.com/posts/999999",
    timeout=10
)
assert missing.status_code == 404

print("Positive and negative responses validated")
```

Expected Result: The existing resource returns 200 and the missing resource returns 404.

Conclusion: Negative API tests verify expected failure behavior.

Experiment 8 – Exception Handling and Timeout

Aim: To distinguish network exceptions from HTTP status failures.

Procedure

1. Wrap the request in try/except.
2. Set a timeout.
3. Handle Timeout.
4. Handle other Requests exceptions.
5. Use raise_for_status for HTTP failures.

Source Code

```
import requests

try:
    response = requests.get(
        "https://jsonplaceholder.typicode.com/posts/1",
        timeout=10
    )
    response.raise_for_status()
    print("Request successful")
    print(response.json())

except requests.exceptions.Timeout:
    print("Request timed out")

except requests.exceptions.RequestException as exc:
    print("Request/network error:", exc)
```

Expected Result: The request succeeds or a meaningful exception is reported.

Conclusion: Exception handling improves reliability and diagnostics.

3.5 BDD with Behave

Behave implements Behavior Driven Development using Gherkin feature files and Python step definitions.

Experiment 9 – Behave REST API Scenario

Aim: To create and execute a Given/When/Then REST API scenario.

Procedure

1. Create features/posts.feature.
2. Write the scenario in Gherkin.
3. Create step definitions.
4. Store the response in context.
5. Run behave.

Source Code

```
# features/posts.feature

Feature: Posts API

    Scenario: Retrieve post 1
        Given the posts API is available
        When I request post 1
        Then the response status should be 200
        And the response should contain post id 1

# features/steps/posts_steps.py

from behave import given, when, then
import requests

BASE_URL = "https://jsonplaceholder.typicode.com"

@given("the posts API is available")
def api_available(context):
    r = requests.get(f"{BASE_URL}/posts/1", timeout=10)
    assert r.status_code == 200

@when("I request post 1")
def request_post(context):
    context.response = requests.get(
        f"{BASE_URL}/posts/1", timeout=10
    )

@then("the response status should be 200")
def status_200(context):
    assert context.response.status_code == 200

@then("the response should contain post id 1")
def contains_id(context):
    assert context.response.json()["id"] == 1

# Run: behave -v
```

Expected Result: Behave maps the Gherkin statements to Python functions and the scenario passes.

Conclusion: BDD separates readable behavior specifications from automation implementation.

Experiment 10 – Scenario Outline, Tags and Hooks

Aim: To execute the same BDD behavior with multiple data values and organize execution.

Procedure

1. Create a Scenario Outline.
2. Add an Examples table.
3. Add a tag such as @smoke.
4. Use before_scenario as a hook.
5. Run only tagged scenarios.

Source Code

```
# features/posts_outline.feature

Feature: Posts API

@smoke
Scenario Outline: Retrieve a post
  When I request post <id>
  Then the response status should be 200

  Examples:
    | id |
    | 1  |
    | 2  |
    | 3  |

# features/environment.py

def before_scenario(context, scenario):
    context.response = None

# Example step:
@when("I request post {id}")
def request_post(context, id):
    import requests
    context.response = requests.get(
        f"https://jsonplaceholder.typicode.com/posts/{id}",
        timeout=10
    )

# Run: behave --tags=@smoke
```

Expected Result: The scenario executes for all example IDs and the smoke tag enables selective execution.

Conclusion: Scenario Outline, Hooks and Tags improve scalability and organization.

Overall Module 3 Lab Conclusion

This report follows the prescribed five Module 3 sections and demonstrates Python fundamentals, HTTP libraries, API/database integration, HTTP response handling and BDD with Behave.

Syllabus Section	Experiments
3.1 Python Basics for REST Automation	1–2
3.2 Python HTTP Libraries	3–4
3.3 API Automation and Database Integration	5–6
3.4 Handling HTTP Responses	7–8
3.5 BDD with Behave	9–10